How the project uses D-Wave

Code walk-through, modes of use, checks, estimates, budget guard, and the 1-second test

`Projects_master_HPQC/scripts/QuantumComputing/Dimod`

Abstract

This document explains, file by file and line by line, how the SK-game campaign talks to D-Wave:

- which Ocean functions it uses, what they can do, and how the code harnesses them;
- the six ways ("modes") of running an annealer in this project;
- what `qpu --check` actually does, and how the QPU time is estimated (three levels, with the formulas);
- how the budget guard (`--budget-seconds`) makes sure a run never exceeds a given QPU time;
- how the results are turned into success probabilities, time-to-solution and the Zephyr/Pegasus comparison.

The code excerpts are read directly from the source files when the document is compiled, so the line numbers are those of the version in the repository. Section 10 gives the exact commands for a test that uses at most one second of QPU time, followed by a plan to spend the 180 s of this month before 4 November.

Contents

1 Where the D-Wave code lives **3**

2 The Ocean SDK: what each package can do, and what we use it for **3**
2.1 `dwave-cloud-client`: connection, solvers, configuration, timing model . . . 3
2.2 `dwave-system`: samplers for the QPU, composites, mock . . . 4
2.3 `dimod`: the problem and the results . . . 5
2.4 `minorminer` and `dwave-graphs`: embeddings and hardware graphs . . . 5
2.5 `dwave.embedding` (inside `dwave-system`): chains in and out . . . 6
2.6 `dwave-samplers` and `dwave-preprocessing` . . . 7

3 The modes of use **7**

4 Life of one QPU call, line by line **8**

5 What `qpu --check` does (and why it costs nothing) **10**

6 How QPU time is estimated: three levels **11**
6.1 Level 1: the rough plan (`main_simulation.py plan`) . . . 11
6.2 Level 2: the solver's timing model (`--check`, budget guard) . . . 11
6.3 Level 3: the charge (`qpu_access_time`) . . . 11

7 The budget guard: `qpu --budget-seconds S` **12**

8 From reads to results **13**

9 How the D-Wave path was tested **15**

10 The one-second test, and a plan for the 180 s **15**
10.1 Test: at most 1 s of QPU time, both topologies 15
10.2 Spending the 180 s before 4 November 15

1 Where the D-Wave code lives

All paths are relative to `Dimod/src/unlearnable_games_explanation/`.

file	role
`skgame/optim/problem.py`	the SK problem W; `to_bqm()` builds the dimod model
`skgame/optim/embedding.py`	topologies (Zephyr, Pegasus), embedding strategies (clique, heuristic), chain statistics, embedding cache
`skgame/optim/backends.py`	`AnnealBackend`: one object per `device:topology:strategy`; builds the sampler, embeds, samples, collects statistics, estimates time
`skgame/optim/quantum.py`	`QuantumAnnealingSolver`: the older single-solver path (`DWaveCliqueSampler`/`EmbeddingComposite`), used by the planner benchmark
`skgame/optim/solvers.py`	`DimodSamplerSolver` (generic dimod wrapper) and the classical dimod samplers
`main_simulation.py`	the campaign: stages `init`, `instances`, `classical`, `embed`, `qpu`/`sim`, `analyze`; the QPU budget guard; the analysis
`tests/test_embedding.py`, `tests/test_campaign.py`	tests of all the above, including the QPU path against D-Wave's mock QPU

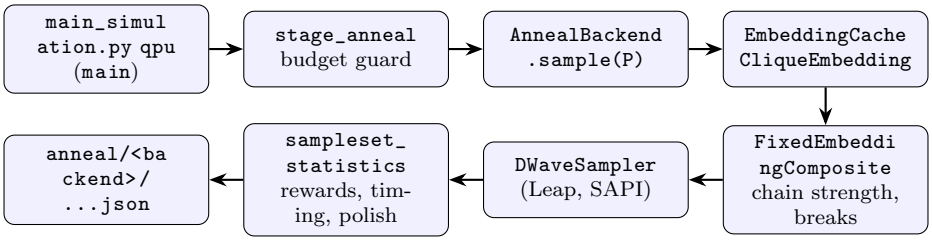


Figure 1: The path of one QPU call through the code.

2 The Ocean SDK: what each package can do, and what we use it for

D-Wave's Python SDK (*Ocean*) is a set of packages, all installed by `cineca/requirements.txt`. For each package: what it offers, then exactly how this project uses it.

2.1 `dwave-cloud-client`: connection, solvers, configuration, timing model

Capabilities.

- it talks to the Leap *Solver API* (SAPI) over HTTPS;
- it reads the token from the configuration file or from the environment (`DWAVE_API_TOKEN`);
- it selects solvers with *feature filters* such as `solver={"topology__type": "zephyr"}`;
- it exposes every solver's `properties`, including the timing data;
- it estimates the cost of a problem (`estimate_qpu_access_time`);
- it submits problems asynchronously;
- it provides the `dwave` command line: `config`, `auth`, `leap`, `solvers`, `ping`, `setup`.

How we use it.

- Through `DWaveSampler` (next subsection), which wraps a cloud-client `Solver`.

- `solver.name` identifies the QPU in the logs and keys the embedding cache, e.g. `Advantage2_system1`.
- `solver.estimate_qpu_access_time` is the second level of our time estimate (§6).
- On the command line: `dwave config create`, `dwave setup --oob`, `dwave solvers -la`, `dwave ping` (`Dimod/LEAP_TOKEN.md`).

2.2 `dwave-system`: samplers for the QPU, composites, mock

Capabilities.

- `DWaveSampler`: a dimod sampler bound to one structured QPU. It exposes `nodelist`/`edgelist` (the working qubits and couplers), `properties`, `parameters` (the accepted keyword arguments) and `to_networkx_graph()`. Its `sample()` accepts `num_reads`, `annealing_time`, `anneal_schedule`, `auto_scale`, `answer_mode`, `h_gain_schedule`, `anneal_offsets`, `programming_thermalization`, `readout_thermalization`, ...
- Composites that turn a structured sampler into one that accepts any problem:
  - `EmbeddingComposite` finds a heuristic embedding on every call;
  - `FixedEmbeddingComposite` uses a given embedding;
  - `LazyFixedEmbeddingComposite` finds one embedding and reuses it;
  - `DWaveCliqueSampler` uses cached all-to-all clique embeddings, scales with the extended J range and computes chain strengths.
- `dwave.system.testing.MockDWaveSampler`: an offline imitation of a QPU of any topology and size, which returns realistic timing fields.

How we use it.

- `DWaveSampler(solver={"topology__type": ...})` picks a Zephyr or Pegasus QPU (code below).
- `to_networkx_graph()` gives the *real* working graph. The embedding is computed on it, so it avoids dead qubits.
- Every QPU call goes through `FixedEmbeddingComposite` with our cached clique embedding. The Zephyr and Pegasus back-ends then go through identical code and differ only in hardware and embedding.
- `DWaveCliqueSampler` and `EmbeddingComposite` remain available in `QuantumAnnealingSolver` (§3).
- `MockDWaveSampler` is used in the tests and in the dry runs.

`skgame/optim/backends.py`, lines 84–114: building the child sampler (QPU, embedded simulator or logical simulator) and reading its hardware graph

```
84      # ------------------------------------------------------------- hardware graph ----
85      @property
86      def child(self):
87          """The structured sampler the embedding maps onto."""
88          if self._child is None:
89              if self.device == "qpu":
90                  from dwave.system import DWaveSampler
91                  s = self.settings
92                  self._child = DWaveSampler(solver=s.solver_name or self.topology.solver_filter)
93              elif self.embedded:
94                  import dimod
95                  from dwave.samplers import SimulatedAnnealingSampler
96                  G = self._target = self.topology.ideal_graph()
```

```
                self._child = dimod.StructureComposite(SimulatedAnnealingSampler(), list(G.nodes), list(G.edges)
    )
            else:
                from dwave.samplers import SimulatedAnnealingSampler
                self._child = SimulatedAnnealingSampler()
        return self._child

    @property
    def target(self):
        """networkx graph of the working qubits and couplers."""
        if self._target is None and self.embedded:
            child = self.child                # building a sim child also sets the ideal graph
            if self._target is not None:
                pass
            elif hasattr(child, "to_networkx_graph"):
                self._target = child.to_networkx_graph()   # real working graph, keeps the topology metadata
            else:
                raise ValueError(f"{self.spec}: cannot read the hardware graph of {type(child).__name__}")
        return self._target
```

2.3 `dimod`: the problem and the results

Capabilities.

- `BinaryQuadraticModel` in SPIN or BINARY form;
- `SampleSet`, which holds the reads: `record.sample`, `record.energy`, `record.num_occurrences`, extra vectors such as `chain_break_fraction`, plus `info` (timing, embedding context). It also offers `aggregate()`, `first` and `resolve()`;
- a common `Sampler` interface (`sample(bqm, **kwargs)`, `parameters`);
- helper composites such as `StructureComposite`, and reference samplers such as `ExactSolver`.

How we use it.

- The SK problem becomes an Ising BQM whose energy is exactly $H(S) = -\sum_{i<j} W_{ij} S_i S_j$.
- `StructureComposite` restricts simulated annealing to the Zephyr or Pegasus graph (the `sim:*:clique` back-ends).
- `ExactSolver` checks the statistics code in the tests.

`skgame/optim/problem.py`, lines 45–50: the SK problem as a dimod BQM ($h = 0$, $J_{ij} = -W_{ij}$)

```
    def to_bqm(self):
        """dimod BinaryQuadraticModel (SPIN) with E = H(S). Lazy import."""
        import dimod
        iu = np.triu_indices(self.N, 1)
        quad = {(int(i), int(j)): -float(self.W[i, j]) for i, j in zip(*iu)}
        return dimod.BinaryQuadraticModel({}, quad, 0.0, dimod.SPIN)
```

2.4 `minorminer` and `dwave-graphs`: embeddings and hardware graphs

Capabilities.

- `minorminer.busclique.find_clique_embedding` returns all-to-all embeddings made of line segments, optimised for chain length and avoiding defects, with an on-disk cache;
- `busgraph_cache(G).largest_clique()` gives the largest K_N the graph supports;
- `minorminer.find_embedding` is the general heuristic;
- `dwave.graphs` (formerly `dwave_networkx`) provides `zephyr_graph`, `pegasus_graph`, coordinate converters and drawing functions.

How we use it.

- The clique strategy and its maximum size (below), which give the `largest_problem` printed by `qpu --check`.
- The heuristic baseline.
- The ideal graphs for the simulator and for the figures in `visualization/embedding_explanation`.

`skgame/optim/embedding.py`, lines 101–115: the all-to-all strategy: embedding and largest clique

```
class CliqueEmbedding(EmbeddingStrategy):
    """Native all-to-all embedding (minorminer.busclique): every chain has the same, minimal-ish
    length (Zephyr: about N/16 + 2 qubits; Pegasus: about N/12 + 2), deterministic and fast.
    This is what DWaveCliqueSampler uses internally."""

    name = "clique"

    def embed(self, N, target):
        from minorminer.busclique import find_clique_embedding
        emb = find_clique_embedding(N, target, use_cache=True)
        return {int(k): [int(q) for q in v] for k, v in emb.items()}

    def max_size(self, target):
        from minorminer.busclique import busgraph_cache
        return len(busgraph_cache(target).largest_clique())
```

`skgame/optim/embedding.py`, lines 157–164: chain statistics; `n_qubits` feeds the readout-time model

```
def chain_stats(embedding: dict) -> dict:
    """Chain lengths of an embedding: the main predictor of chain breaks and of the energy
    compression imposed by the chain couplings."""
    if not embedding:
        return {"n_qubits": 0, "chain_len_mean": float("nan"), "chain_len_max": 0, "chain_len_min": 0}
    L = np.array([len(c) for c in embedding.values()])
    return {"n_qubits": int(L.sum()), "chain_len_mean": float(L.mean()),
            "chain_len_max": int(L.max()), "chain_len_min": int(L.min())}
```

2.5 `dwave.embedding` (inside `dwave-system`): chains in and out

Capabilities.

- `embed_bqm` maps a logical BQM onto qubits: it splits each J_{ij} over the couplers between two chains and adds $-c$ on the chain couplers;
- `unembed_sampleset` maps reads back to logical spins;
- `chain_strength.uniform_torque_compensation` gives $c = p\, J_{\mathrm{rms}}\sqrt{\bar{d}}$;
- `chain_breaks`: `majority_vote`, `weighted_random`, `discard`, `MinimizeEnergy`.

How we use it.

- The chain strength is `partial(uniform_torque_compensation, prefactor=p)`, with $p =$ `chain_strength_prefactor`.
- The chain-break method is chosen by name.
- `MinimizeEnergy` has to be wrapped in a named function, because `unembed_sampleset` reads `__name__`. Testing revealed this.

skgame/optim/backends.py, lines 145–156: the keyword arguments of one call

```
145     def sample_kwargs(self, bqm, emb) -> dict:
146         s = self.settings
147         if self.device == "qpu":
148             kw = {"num_reads": s.num_reads, "annealing_time": s.annealing_time}
149         else:
150             kw = {"num_reads": s.sim_num_reads, "num_sweeps": s.sim_num_sweeps}
151         if self.embedded:
152             from dwave.embedding.chain_strength import uniform_torque_compensation
153             kw["chain_strength"] = partial(uniform_torque_compensation, prefactor=s.chain_strength_prefactor)
154             kw["chain_break_method"] = _chain_break_method(s.chain_break_method, bqm, emb)
155             kw["return_embedding"] = True
156         return kw
```

skgame/optim/backends.py, lines 197–207: chain-break resolution by name

```
197 def _chain_break_method(name: str, bqm, emb):
198     from dwave.embedding import chain_breaks as cb
199     if name == "minimize_energy":
200         resolver = cb.MinimizeEnergy(bqm, emb)
201 
202         def minimize_energy(samples, chains):    # unembed_sampleset needs a __name__
203             return resolver(samples, chains)
204         return minimize_energy
205     if name in ("majority_vote", "weighted_random", "discard"):
206         return getattr(cb, name)
207     raise ValueError(f"chain_break_method must be one of {CHAIN_BREAK_METHODS}")
```

2.6 `dwave-samplers` and `dwave-preprocessing`

Capabilities.

- classical samplers with the dimod interface: `SimulatedAnnealingSampler`, `TabuSampler`, `SteepestDescentSolver`, `TreeDecompositionSolver`, `RandomSampler`;
- `SpinReversalTransformComposite` (gauge transforms) and other pre-processing.

How we use it.

- SA, tabu and steepest descent are classical baselines (`solvers.py`).
- SA is also the engine of the `sim:logical` and `sim:*:clique` back-ends.
- Spin-reversal transforms are discussed as a mitigation (`explanation_project.pdf`, noise section) but are not yet wired in.

3 The modes of use

mode	what runs	when to use it	QPU time / token
`sim:logical`	SA on K_N directly (no hardware graph)	reference for the annealing pipeline	none / no
`sim:zephyr:clique`, `sim:pegasus:clique`	the *embedded* problem, chains included, solved by SA on the ideal graph (`StructureComposite`)	test the whole embedding path; classical view of chain effects	none / no
mock QPU (tests)	`MockDWaveSampler` in place of `DWaveSampler`, same code path as the QPU	tests, dry runs	none / no

mode	what runs	when to use it	QPU time / token
`qpu:zephyr:clique`, `qpu:pegasus:clique`	real QPU through `FixedEmbeddingComposite` + our clique embedding	the campaign	yes / yes
`qpu:*:minorminer`	real QPU with a heuristic embedding	baseline only	yes / yes
`QuantumAnnealingSolver`	`DWaveCliqueSampler` (clique, extended J range) or `EmbeddingComposite` (heuristic each call)	the planner benchmark (`skgame/optim/benchmark.py`)	yes / yes

`skgame/optim/quantum.py`, lines 49–58: the single-solver path: `DWaveCliqueSampler` or `EmbeddingComposite`

```
49  def make_sampler(self):
50      if self.use_simulator:
51          from dwave.samplers import SimulatedAnnealingSampler
52          return SimulatedAnnealingSampler()
53      if self.clique:
54          from dwave.system import DWaveCliqueSampler
55          return DWaveCliqueSampler(solver=self.solver_name) if self.solver_name else DWaveCliqueSampler()
56      from dwave.system import DWaveSampler, EmbeddingComposite
57      base = DWaveSampler(solver=self.solver_name) if self.solver_name else DWaveSampler()
58      return EmbeddingComposite(base)
```

The campaign uses `FixedEmbeddingComposite` rather than `DWaveCliqueSampler` so that both topologies run *identical* code: same embedding strategy, same chain-strength rule, same scaling. Their results then differ only because of the hardware.

4 Life of one QPU call, line by line

`skgame/optim/backends.py`, lines 158–185: `AnnealBackend.sample`: embed, call, collect

```
158 def sample(self, P: PlannerProblem, rng=None) -> dict:
159     from dwave.system import FixedEmbeddingComposite
160     rng = np.random.default_rng(rng)
161     bqm = P.to_bqm()
162     if self.embedded:
163         emb = self.embedding(P.N)
164         sampler = FixedEmbeddingComposite(self.child, emb)
165     else:
166         emb, sampler = None, self.child
167     kw = self.sample_kwargs(bqm, emb)
168     params = getattr(self.child, "parameters", {}) or {}
169     if self.device == "sim" and "seed" in params:
170         kw["seed"] = int(rng.integers(2**31))
171     t0 = time.perf_counter()
172     ss = sampler.sample(bqm, **kw)
173     ss.resolve()
174     wall = time.perf_counter() - t0
175     out = sampleset_statistics(ss, P, wall, polish=self.settings.polish)
176     info = out["info"]
177     info.update(backend=self.spec, solver=self.solver_id(), settings=asdict(self.settings),
178                 sample_kwargs={k: v for k, v in kw.items() if not callable(v) and k != "seed"})
179     if emb is not None:
180         info["embedding"] = chain_stats(emb)
181         ctx = ss.info.get("embedding_context", {})
182         cs = ctx.get("chain_strength")
183         info["chain_strength"] = float(cs) if np.isscalar(cs) else None
184         info["max_abs_logical_coupling"] = float(np.max(np.abs(P.W)))
185     return out
```

What happens, in order:

1. `P.to_bqm()`: the SK problem as an Ising BQM.

2. `self.embedding(P.N)`: the clique embedding of K_N on the solver's real graph, read from `embeddings/<solver>/clique/N....json` or computed and cached.

3. `FixedEmbeddingComposite(child, emb)`. When it samples, it computes c with the chain-strength callable, embeds the problem (`embed_bqm`) and sends it to `DWaveSampler`. That sampler submits it to SAPI with `num_reads` and `annealing_time`; `auto_scale` is on by default and divides by the largest coefficient. The reads come back and are unembedded with the chain-break method.

4. `ss.resolve()` waits for the asynchronous answer.

5. `sampleset_statistics` recomputes every reward from the spins: it is exact whatever scaling was applied. It aggregates identical reads, records `info["timing"]` (in μs) and the read-averaged chain-break fraction, and optionally polishes every distinct read with greedy descent.

6. The embedding statistics, the chain strength actually used (from `embedding_context`) and $\max|W_{ij}|$ are added.

`skgame/optim/backends.py`, lines 210–240: statistics of a sample set

```
210 def sampleset_statistics(ss, P: PlannerProblem, wall: float, polish: bool = False) -> dict:
211     """Distinct rewards and their read counts (best first), best spins, QPU timing, chain breaks.
212 
213     Rewards are recomputed from the spins, so they are exact in units of R_N whatever scaling
214     the sampler applied. With 'polish', every distinct read is also relaxed by greedy single-flip
215     ascent (a classical post-processing step that repairs the residual errors of chain breaks
216     and of analogue noise) and reported separately.
217     """
218     agg = ss.aggregate()
219     cols = [list(agg.variables).index(i) for i in range(P.N)]
220     S = np.asarray(agg.record.sample, float)[:, cols]
221     counts = np.asarray(agg.record.num_occurrences, int)
222     rewards = np.atleast_1d(P.reward(S))
223     order = np.argsort(-rewards)
224     info = {"wall_seconds": wall, "num_reads": int(counts.sum())}
225     timing = ss.info.get("timing")
226     if timing:
227         info["timing_us"] = {k: float(v) for k, v in timing.items() if np.isscalar(v)}
228     if "chain_break_fraction" in ss.record.dtype.names:
229         info["chain_break_fraction_mean"] = float(np.average(ss.record.chain_break_fraction,
230                                                              weights=ss.record.num_occurrences))
231     out = {"rewards": rewards[order].tolist(), "counts": counts[order].tolist(),
232            "best_reward": float(rewards[order[0]]), "best_S": _spin_string(S[order[0]]), "info": info}
233     if polish:
234         from .solvers import _greedy
235         Sp = np.array([_greedy(P.W, s.copy())[0] for s in S])
236         rp = np.atleast_1d(P.reward(Sp))
237         op = np.argsort(-rp)
238         out.update(polished_rewards=rp[op].tolist(), polished_counts=counts[op].tolist(),
239                    polished_best_reward=float(rp[op[0]]), polished_best_S=_spin_string(Sp[op[0]]))
240     return out
```

The `seed` lesson. Samplers differ in which arguments they accept. The QPU rejects unknown ones *server-side*, so a Python `try/except TypeError` would never catch the error. Both wrappers therefore pass `seed` only to samplers that declare it in `parameters`:

`skgame/optim/solvers.py`, lines 379–388: passing `seed` only where it is accepted

```
379         # only samplers that declare a 'seed' parameter get one: the QPU rejects unknown kwargs
380         # server side (not with a TypeError), so trying and catching is not enough there
381         params = getattr(self._sampler, "parameters", None)
382         if params is None or "seed" in params:
383             kw.setdefault("seed", int(rng.integers(2**31)))
384         try:
385             ss = self._sampler.sample(bqm, **kw)
386         except TypeError:
387             kw.pop("seed", None)
388             ss = self._sampler.sample(bqm, **kw)
```

5 What `qpu --check` does (and why it costs nothing)

`python main_simulation.py qpu --out DIR --check` runs, for each QPU back-end of `config.json`:

1. `make_backend` builds an `AnnealBackend`.
2. `be.largest_problem()` builds `DWaveSampler(solver={"topology__type": ...})`. This contacts SAPI to choose an online solver of that family and download its properties: **metadata only, no problem is submitted**. The method then reads the working graph and calls `busgraph_cache(G).largest_clique()`.
3. The list of calls is built. N above the largest clique is skipped; already-saved results are skipped (resume).
4. `_estimate_qpu_time` computes, for each N, the clique embedding on the real graph and the number of qubits it uses, then evaluates `solver.estimate_qpu_access_time`: **a local formula, no QPU**.

`main_simulation.py`, lines 414–429: the `--check` estimate

```
414 def _estimate_qpu_time(be, todo, log) -> None:
415     """QPU access time still to spend, from the solver's own timing model (no QPU time used)."""
416     if be.device != "qpu":
417         log("[anneal] not a QPU back-end: nothing to estimate")
418         return
419     per_N = {}
420     for N in sorted({u[3] for u in todo}):
421         per_N[N] = be.estimate_access_time(N)
422         if per_N[N] is None:
423             log("[anneal] the solver exposes no timing model")
424             return
425         log(f"[anneal]   N={N:4d}: ~{per_N[N] * 1e3:.0f} ms per call")
426     total = sum(per_N[u[3]] for u in todo)
427     log(f"[anneal] estimated QPU access time for the {len(todo)} remaining calls: {total:.1f} s")
```

`skgame/optim/backends.py`, lines 187–194: one call's estimate from the solver's timing model

```
187     def estimate_access_time(self, N: int) -> float | None:
188         """Seconds of QPU access time for one call, from the solver's timing model (no QPU use)."""
189         solver = getattr(self.child, "solver", None)
190         est = getattr(solver, "estimate_qpu_access_time", None)
191         if est is None:
192             return None
193         n_qubits = chain_stats(self.embedding(N))["n_qubits"]
194         return est(n_qubits, num_reads=self.settings.num_reads, annealing_time=self.settings.annealing_time) / 1
     e6
```

back-end	solver	largest N	calls (`full`)	estimated time
`qpu:zephyr:clique`	`Advantage2_system1`	104	260 ($N \leq 104$)	52.3 s
`qpu:pegasus:clique`	`Advantage_system4`	177	400	105.0 s

Your check (27 October). On the real Zephyr chip the largest all-to-all problem is $N = 104$, far below the 184 of the ideal graph. Defects remove intact line segments (`explanation_project.pdf`, embedding step 10).

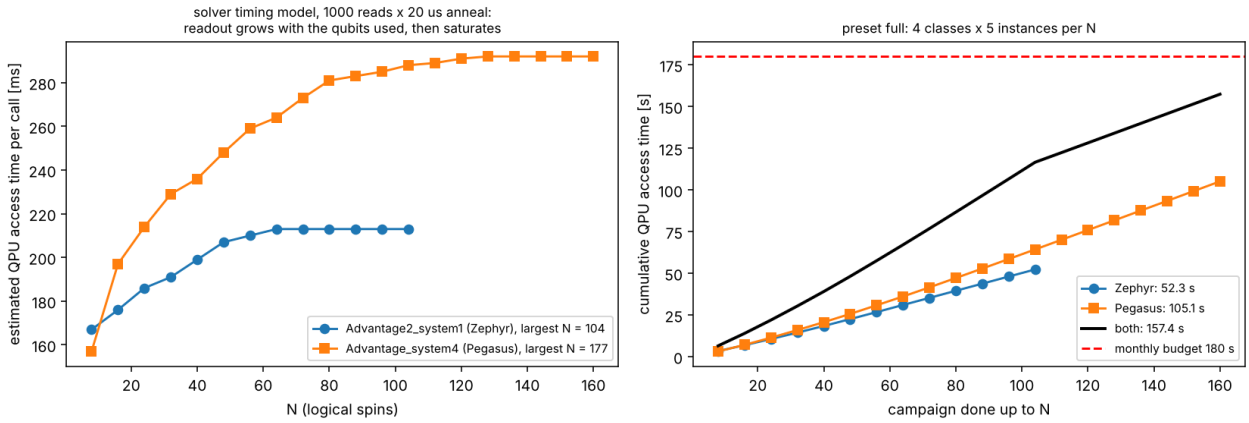


Figure 2: Left: estimated access time per call, from your `--check`. Right: cumulative budget of the `full` campaign: 52.3 s (Zephyr) + 105.0 s (Pegasus) = 157.3 s, against 180 s.

6 How QPU time is estimated: three levels

6.1 Level 1: the rough plan (`main_simulation.py plan`)

For a first idea, before any connection:

$$t_{\text{call}} \approx t_{\text{prog}} + n_{\text{reads}}\,(t_a + t_{\text{ro}} + t_d) = 15\,\text{ms} + 1000\,(20 + 100 + 20)\,\mu\text{s} = 155\,\text{ms}. \tag{1}$$

The constants are generic, and your check shows they **underestimate**: the real per-call times are 157–292 ms. Use level 1 only to compare designs, never to spend.

6.2 Level 2: the solver's timing model (`--check`, budget guard)

D-Wave publishes, in each solver's `properties["problem_timing_data"]`, the parameters of a model that `estimate_qpu_access_time` evaluates. Reading its source (`dwave-cloud-client` 0.14.8) gives

$$t_{\text{access}} = \underbrace{t^{\text{typ}}_{\text{prog}} + t^{\text{therm}}_{\text{prog}}}_{\text{programming}} + n_{\text{reads}}\Big(t_a + t_{\text{ro}}(n_{\text{qubits}}) + t_{\text{delay}} + \max(t^{\text{therm}}_{\text{ro}}, t_{\text{decorr}})\Big), \tag{2}$$

$$t_{\text{ro}}(n_{\text{qubits}}) = \text{piecewise-linear interpolation (often in log–log) of a measured curve.} \tag{3}$$

Two consequences, both visible in your check:

- the cost grows with N, because more qubits mean a longer readout, and then **saturates**;
- Pegasus costs more per call than Zephyr (292 vs 213 ms at large N), because its chains use more qubits.

Above all, the time is **proportional to `num_reads`** apart from the programming term. Halving the reads roughly halves the budget.

6.3 Level 3: the charge (`qpu_access_time`)

After each call the QPU returns its timing. `qpu_access_time` is what Leap charges. It is saved in every result file together with the level-2 estimate (`estimated_access_s`), and `qpu --spent` sums them and prints the ratio actual/estimate:

`main_simulation.py`, lines 396–411: `qpu --spent`: what was actually charged

```
396 def qpu_spent(out: Path, log=print) -> dict:
397     """QPU access time already spent in a campaign, per back-end, from the saved results
```

```
    (the value D-Wave charged for each call), with the ratio to the prediction."""
    rows = {}
    for f in sorted((out / "anneal").glob("qpu_*/*/*.json")) if (out / "anneal").exists() else []:
        info = json.loads(f.read_text())["info"]
        r = rows.setdefault(f.parent.parent.name, {"calls": 0, "spent_s": 0.0, "est_s": 0.0})
        r["calls"] += 1
        r["spent_s"] += info.get("timing_us", {}).get("qpu_access_time", 0.0) / 1e6
        r["est_s"] += info.get("estimated_access_s", 0.0)
    total = sum(r["spent_s"] for r in rows.values())
    for name, r in rows.items():
        ratio = f"{r['spent_s'] / r['est_s']:.2f}" if r["est_s"] else "n/a"
        log(f"[spent] {name:22s} {r['calls']:4d} calls  {r['spent_s']:8.3f} s  (actual/estimate = {ratio})")
    log(f"[spent] total QPU access time in {out}: {total:.3f} s")
    return rows
```

7 The budget guard: `qpu --budget-seconds S`

`main_simulation.py`, lines 312–343: budget object and per-call estimate

```
class QPUBudget:
    """Hard cap on the QPU access time spent by one command (all back-ends together).

    Before each call the cost is predicted with the solver's own timing model
    (`estimate_qpu_access_time`) and inflated by a safety factor; the call is made only if
    spent + inflated estimate <= limit. The factor starts at `safety` and grows if real
    charges come out larger than predicted (factor >= 1.1 x worst observed actual/estimate).
    After each call the REAL `qpu_access_time` returned by the QPU is added to `spent`.
    """

    def __init__(self, limit_s: float | None, safety: float = 1.5):
        self.limit, self.safety, self.spent, self.worst_ratio = limit_s, safety, 0.0, 0.0

    def factor(self) -> float:
        return max(self.safety, 1.1 * self.worst_ratio)

    def allows(self, estimate_s: float) -> bool:
        return self.limit is None or self.spent + self.factor() * estimate_s <= self.limit

    def charge(self, actual_s: float, estimate_s: float) -> None:
        self.spent += actual_s
        if estimate_s > 0 and actual_s > 0:
            self.worst_ratio = max(self.worst_ratio, actual_s / estimate_s)


def _call_estimate(be, N: int, cfg: CampaignConfig) -> float:
    """Predicted QPU access time of one call [s]. Without a timing model (mock QPU) a
    deliberately pessimistic formula: 20 ms programming + reads x (anneal + 300 us)."""
    est = be.estimate_access_time(N) if be.device == "qpu" else 0.0
    if est is None:
        est = 0.020 + cfg.num_reads * (cfg.annealing_time * 1e-6 + 300e-6)
    return float(est)
```

`main_simulation.py`, lines 346–394: the anneal stage: guard before every call, real charge after it

```
def stage_anneal(out: Path, cfg: CampaignConfig, spec: str = "sim:logical", keys=None, max_calls=None,
                 solver_name=None, check_only=False, log=print, retries=3, backend=None,
                 budget: QPUBudget | None = None) -> None:
    be = backend or make_backend(out, cfg, spec, solver_name)
    limit = be.largest_problem()
    log(f"[anneal] backend={be.spec} solver={be.solver_id()} largest_problem={limit}")
    units = list(anneal_units(out, cfg, keys))
    too_big = sorted({u[3] for u in units if limit is not None and u[3] > limit})
    if too_big:
        log(f"[anneal] skipping N={too_big}: above the largest embeddable clique ({limit})")
    todo = [u for u in units if (limit is None or u[3] <= limit)
            and not anneal_path(out, be.label, u[0], u[3], u[4]).exists()]
    log(f"[anneal] {len(units)} calls in the campaign, {len(todo)} still to do")
    if check_only:
        _estimate_qpu_time(be, todo, log)
        return
    total_access = 0.0
    budget = budget or QPUBudget(None)
```

```
    for n, (key, eps, J0, N, b) in enumerate(todo):
        if max_calls is not None and n >= max_calls:
            log(f"[anneal] stopped after --max-calls {max_calls}; rerun to continue")
            break
        est = _call_estimate(be, N, cfg)
        if not budget.allows(est):
            log(f"[anneal] budget: spent {budget.spent:.3f}s of {budget.limit:.3f}s; next call (N={N}) estimated
 "
                f"{est:.3f}s x safety {budget.factor():.2f} would exceed it -> stopped (rerun to continue)")
            break
        J = ensure_instances(out, cfg, key, eps, J0, N)
        P = PlannerProblem(J[b])
        for attempt in range(retries + 1):
            try:
                res = be.sample(P, rng=np.random.SeedSequence([cfg.seed, 3, N, b]))
                break
            except Exception as exc:                    # network hiccups, solver busy, ...
                if attempt == retries:
                    raise
                log(f"[anneal] {key} N={N} i={b}: {type(exc).__name__}: {exc}; retry in {2 ** (attempt + 1)}s")
                time.sleep(2 ** (attempt + 1))
        res.update(key=key, eps=eps, J0=J0, N=N, instance=b, backend=be.spec)
        res["info"]["estimated_access_s"] = est
        _write_json(anneal_path(out, be.label, key, N, b), res)
        acc = res["info"].get("timing_us", {}).get("qpu_access_time", 0.0) / 1e6
        budget.charge(acc if acc > 0 else est, est)
        total_access += acc
        cbf = res["info"].get("chain_break_fraction_mean")
        log(f"[anneal] {n + 1:4d}/{len(todo)} {be.label} {key:10s} N={N:4d} i={b} best={res['best_reward']:.5f}
 "
            f"chain_breaks={'-' if cbf is None else f'{cbf:.3f}'} qpu_access={acc * 1e3:.1f}ms (est {est * 1e3
:.0f}ms) "
            f"(total {total_access:.3f}s)")
```

Rule: before each call, with $\hat{t}$ the level-2 estimate,

$$\text{call allowed} \iff t_{\text{spent}} + f\,\hat{t} \le S, \qquad f = \max\left(1.5,\ 1.1 \cdot \max_{\text{past calls}} t_{\text{actual}}/\hat{t}\right). \tag{4}$$

- The guard starts with a 50% safety margin. If the real charges turn out larger than the model, the margin grows automatically.
- After every call t_{spent} grows by the *real* `qpu_access_time`.
- The budget covers all back-ends of one command (Zephyr, then Pegasus).
- When a call is refused, the command stops cleanly. Everything done so far is saved, and the same command continues later (resume).
- Without a timing model (mock QPU) the estimate is a deliberately pessimistic formula: $20\,\text{ms} + n_{\text{reads}}(t_a + 300\,\mu\text{s})$.

It is tested in `tests/test_campaign.py`: the rule itself, a 0.01 s budget that refuses every call, and a 1 s budget that completes and records estimates.

What the guard cannot guarantee. It caps the time *this code* spends, using D-Wave's own numbers. It cannot stop a call once submitted, and it does not see time spent by other programs or other users of the same Leap project. The Leap dashboard remains the authority on what is left.

8 From reads to results

`main_simulation.py`, lines 467–498: per back-end metrics in `collect`

```
            for s, recs in ann.items():
                done = [(b, r) for b, r in enumerate(recs) if r is not None]
                if not done:
                    continue
```

```
                p = np.array([np.sum(np.array(r["counts"])[_success(r["rewards"], ref[b])]) / np.sum(r["counts"])
                              for b, r in done])
                best_res = np.array([(ref[b] - r["best_reward"]) / abs(ref[b]) for b, r in done])
                mean_res = np.array([np.average((ref[b] - np.array(r["rewards"])) / abs(ref[b]), weights=r["counts"])
                                     for b, r in done])
                t_read = np.median([_time_per_read(r) for b, r in done])
                acc = [r["info"].get("timing_us", {}).get("qpu_access_time", np.nan) / 1e6 for b, r in done]
                cbf = [r["info"].get("chain_break_fraction_mean", np.nan) for b, r in done]
                extra = {}
                if all("polished_rewards" in r for b, r in done):
                    pp = np.array([np.sum(np.array(r["polished_counts"])[_success(r["polished_rewards"], ref[b])])
                                   / np.sum(r["polished_counts"]) for b, r in done])
                    extra.update(p_success_polished=float(np.median(pp)),
                                 residual_polished=float(np.median([(ref[b] - r["polished_best_reward"]) / abs(ref[b])
                                                                    for b, r in done])))
                emb = [r["info"].get("embedding") for b, r in done]
                if all(emb):
                    extra.update(chain_len_mean=float(np.mean([e["chain_len_mean"] for e in emb])),
                                 chain_len_max=int(max(e["chain_len_max"] for e in emb)),
                                 n_qubits=int(np.mean([e["n_qubits"] for e in emb])),
                                 chain_strength=_nan_to_none(float(np.mean([r["info"].get("chain_strength") or np.nan
                                                                            for b, r in done]))))
                rows.append(dict(base, kind="anneal", method=f"anneal_{s}", regime="", n_done=len(done), **extra,
                                 p_success_median=float(np.median(p)), p_success_mean=float(p.mean()),
                                 p_best_found=float(np.mean(best_res <= TOL)),
                                 residual_median=float(np.median(best_res)), residual_mean_read=float(np.median(mean_res)),
                                 seconds_median=t_read, tts99=_tts(p.mean(), t_read),
                                 qpu_access_s=float(np.nansum(acc)), chain_break_fraction=_nan_to_none(float(np.mean(cbf)))))
```

For each problem class, N and instance b, with reference R_{ref} (exact for $N \le 24$; otherwise the best of the classical solvers and of every annealer read, polished reads included):

$$p_{\text{success}} = \frac{\#\{\text{reads with } R \ge R_{\text{ref}} - 10^{-9}\}}{n_{\text{reads}}} \quad \text{(per read)}, \tag{5}$$

$$\text{residual} = \frac{R_{\text{ref}} - R_{\text{best read}}}{|R_{\text{ref}}|}, \tag{6}$$

$$t_{\text{read}} = \frac{t_{\text{sampling}}}{n_{\text{reads}}} \quad (t_{\text{sampling}} = \texttt{qpu_sampling_time}), \tag{7}$$

$$\text{TTS}_{99} = t_{\text{read}} \frac{\ln 0.01}{\ln(1-p)}. \tag{8}$$

`main_simulation.py`, lines 517–532: time per read and TTS_{99}

```
def _time_per_read(r) -> float:
    """QPU: sampling time per read (anneal + readout + delay); simulator: wall time per read."""
    t = r["info"].get("timing_us", {})
    n = r["info"]["num_reads"]
    if "qpu_sampling_time" in t:
        return t["qpu_sampling_time"] / 1e6 / n
    return r["info"]["wall_seconds"] / n


def _tts(p: float, t: float) -> float:
    """Time to reach the reference with 99 % confidence: t ln(0.01) / ln(1 - p)."""
    if p >= 1:
        return float(t)
    if p <= 0:
        return INF
    return float(t * math.log(0.01) / math.log1p(-p))
```

`compare_backends` ranks the back-ends of the same device for each (class, N): higher median p_{success} wins, ties go to the lower residual. The output is `summary/backend_comparison.csv` and `.png`.

9 How the D-Wave path was tested

- `test_embedding.py`: clique embeddings are valid on both topologies (every chain connected, chains disjoint, every pair linked); Zephyr chains are shorter at production size; the cache round-trips; back-end specs parse; the embedded simulator finds the exact ground state, and the chain strength equals $p\,J_{\mathrm{rms}}\sqrt{N-1}$ computed independently.
- `test_campaign.py`: every read is kept and `seed` is skipped where unsupported (with `ExactSolver`); the whole campaign runs end to end and resumes; **both QPU back-ends run against** `MockDWaveSampler` with Zephyr and Pegasus topologies, using the real `DWaveSampler` code path from `solver={topology__type}` onwards; the budget guard (rule, refusal, completion, `--spent`) is checked.
- What the mock cannot test: the real timing values, the real noise, SAPI errors and Leap accounting. That is what the one-second test below is for.

10 The one-second test, and a plan for the 180 s

10.1 Test: at most 1 s of QPU time, both topologies

A tiny separate campaign: $N = 8$ and 16, one instance, one problem class, 100 reads per call, both QPUs. That makes 4 calls. At 100 reads each the timing model gives a few tens of ms per call, so the whole test costs a fraction of a second. The guard makes **1 s a hard ceiling** anyway.

```
cd ...\Dimod\src\unlearnable_games_explanation      # (PowerShell, .venv active)
$T = "results\qpu_test"
python main_simulation.py init --preset smoke --out $T --Ns 8,16 --n-instances 1 --regimes
    sg_satisficing --num-reads 100
python main_simulation.py qpu --out $T --check                  # predicted cost: spends
    nothing
python main_simulation.py qpu --out $T --budget-seconds 1       # the test: <= 1 s,
    normally ~0.1-0.2 s
python main_simulation.py qpu --out $T --spent                  # what Leap charged, and
    actual/estimate
python main_simulation.py classical --out $T                    # seconds, for the
    reference optimum
python main_simulation.py analyze --out $T                      # success per read, chain
    breaks
```

What to check in the output:

1. every call prints `qpu_access=...ms (est ...ms)`: real vs predicted;
2. `--spent`: the total, well below 1 s, and `actual/estimate` close to 1;
3. `chain_breaks`: should be 0 or tiny at $N = 8, 16$;
4. the Leap dashboard shows the same usage.

10.2 Spending the 180 s before 4 November

Your check gives 157.3 s for the full campaign on both topologies at 1000 reads. That leaves about 22 s of margin, *if* the real charges match the model, which the test will show. Two safe options:

plan	QPU time	comment
`full`, 1000 reads, both QPUs	≈ 157 s	as designed; run with `--budget-seconds 170` so it can never pass 170 s
`full`, 500 reads (`init ... --num-reads 500`)	≈ 80 s	time $\propto$ reads; leaves ≈ 90 s for a chain-strength scan at the N where success drops

```
$F = "results\full"
python main_simulation.py init --preset full --out $F --force          # (add --num-reads
    500 for option 2)
python main_simulation.py qpu --out $F --check
python main_simulation.py qpu --out $F --budget-seconds 170            # stops by itself
    before 170 s
python main_simulation.py qpu --out $F --spent
```

The QPU stage does not need the classical stage. That can run before or after, on the laptop or on Leonardo, and `analyze` combines them. If the run stops on the budget, it has saved everything done so far, and `--spent` shows the total.

References

- D-Wave, Ocean documentation: https://docs.dwavequantum.com/en/latest/ocean/; Leap authorisation: https://docs.dwavequantum.com/en/latest/ocean/leap_authorization.html.
- D-Wave, QPU timing and `estimate_qpu_access_time`: `dwave.cloud.solver.StructuredSolver` (source read for §6, dwave-cloud-client 0.14.8).
- Boothby, King, Raymond, *Fast clique minor generation in Chimera qubit connectivity graphs*, Quantum Inf. Process. 15, 495 (2016): the clique embedding.
- Cai, Macready, Roy, *A practical heuristic for finding graph minors*, arXiv:1406.2741 (2014): minorminer.
- Cineca, Quantum Computing as a Service: https://www.quantumcomputinglab.cineca.it/en/resources.
- Project: `Dimod/explanation_project.pdf` (SK game, algorithms, embedding, chains, noise), `Dimod/LEAP_TOKEN.md`, `README_LOCAL.md`, `README_CINECA.md`.